

Java 面向对象进阶编程（第五次作业）提交报告

> ** ⚠️ 提交要求重要提示: **

>

> 1. 每道题目的****核心源码****需要在下方模板中以 Java 代码块形式给出。

> 2. 所有作业形成的独立的 Java 源码文件（及目录结构），请****打包为一个 ZIP 文件夹后以附件形式上传****系统。

****【基本信息】****

- 姓名：谢锦潇
- 学号：2535010414
- 班级：计算机4班
- 提交日期：2026 年 4 月 11 日 23:59:59
- 开发环境：JDK 25 + vscode

📦 题目一：物流公司的运费结算系统（抽象类与多态）

****1. 代码****（重点粘贴 `Parcel` 抽象类及 `AccountingSystem` 中的多态结算方法）********

```
java
```

```
abstract class Parcel{  
  
    protected double weight;  
  
    abstract public double calculateCost();  
}
```

```

    public Parcel(double weight){
        this.weight=weight;
    }

    public Parcel(){

    }

    public void printInfo()
    {
        System.out.println("Weight: " + weight);
        System.out.println("Cost: " + calculateCost());
    }

}

class AccountingSystem{
    public static double calculatteRevenue(Parcel[]parcels)
    {
        double totalARevenue=0.0;
        for(Parcel p:parcels)
        {
            totalARevenue+=p.calculateCost();
        }
        return totalARevenue;
    }
}

```

```

```java

```

// 请在此处粘贴核心代码

// 注意：需展示 abstract 关键字的使用，以及财务类中是如何通过包裹数组循环计算总运费的

...

**\*\*2. 运行结果截图： \*\***

```
Weight: 5.0
Cost: 50.0

Weight: 3.0
Cost: 110.0

Total Revenue: 160.0
>
```

**\*\*3. 📝 思考题（必答）： \*\***

在这里，我们为什么让两种不同快递继承 `Parcel` 抽象类，而不是去实现一个接口？

> [请在此处写下你的理解]

> 因为可以复用 printInfo() 代码

>

### 🏠 题目二：米家智能家居控制中心（接口与多态）

**\*\*1. 核心代码展示（重点粘贴 `SmartDevice` 接口及 `SmartHomeHub` 类的一键离家方法）： \*\***

```
java

interface SmartDevice{

 void turnOn();

 void turnOff();

}

class SmartHomeHub{

 public static void leaveHome(SmartDevice[] devices)

 {

 for(SmartDevice d:devices)

 {

 d.turnOff();

 }

 }

}
```

```
```java
```

```
// 请在此处粘贴测试类中的核心代码
```

```
// 注意：必须体现创建 SmartDevice[] 数组，以及在 SmartHomeHub 中统一调用  
turnOff() 的过程
```

```
```
```

**\*\*2. 运行结果截图：\*\***

```
Smart Bulb is now OFF
Smart Curtain is now OFF
Smart AC is now OFF
>
```

**\*\*3. 📄 思考题（必答）：\*\***

灯泡、空调、窗帘明明属于完全不同的家电种类，它们没有共同的“本质血缘”，为什么我们依然能把它装进同一个数组统一管理？这体现了接口的什么作用？

> [请在此处写下你的理解]

> 可以实现多态

>

**### ✂ 题目三：RPG 游戏的防空塔防系统（单继承与多重契约）**

**\*\*1. 核心代码展示（重点粘贴 `PegasusRider` 或 `Gargoyle` 的类定义，以及 `AntiAirTower` 的锁定方法）：\*\***

java

```
class Gargoyle extends Monster implements Flyable{

 public Gargoyle(String name)

 {

 super(name);

 }

 public void fly()

 {

 System.out.println(name+" is flying.");

 }

}

class AntiAirTower{

 public void lockAndFire(Flyable target)
```

```

 {
 System.out.println("Anti-Air Tower locks onto target.");
 target.fly();
 System.out.println("Anti-Air Tower fires at the target!");
 }
}

```

```
```java
```

```
// 请在此处粘贴核心代码
```

```
// 注意：需展示子类是如何同时使用 extends 继承和 implements 实现的
```

```
```
```

**\*\*2. 运行结果截图： \*\***

```

Anti-Air Tower locks onto target.
Gargantua is flying.
Anti-Air Tower fires at the target!
Anti-Air Tower locks onto target.
Pegasus Rider is flying.
Anti-Air Tower fires at the target!
>
>
>

```

### 🔄 题目四：电子垃圾回收站的高级提炼（向下转型与 instanceof）

**\*\*1. 核心代码展示（重点粘贴 `RecycleLine` 处理方法中安全转型的核心判断逻辑）：**

**\*\***

```
java
```

```
class RecycleLine{
 static public void processWaste(Ewaste item)
 {
 item.crush();
 if(item instanceof Phone)
 {
 ((Phone) item).extractLithium();
 }
 else if(item instanceof Motherboard)
 {
 ((Motherboard) item).extractGold();
 }
 }
}
```

```
```java
```

```
// 请在此处粘贴核心代码
```

```
// 注意：必须展示你是如何用 instanceof 判断对象身份，并执行强制类型转换（向下  
塑型）去调用高级技能的
```

```
```
```

**\*\*2. 运行结果截图： \*\***

```
Crushing e-waste into smaller pieces...
Extracting lithium from crushed phone...
Crushing e-waste into smaller pieces...
Extracting gold from crushed motherboard...
> **3.
```

 **思考题（必答）：** \*\*

如果不加 `instanceof` 作为安全验证，直接盲目使用强制类型转换 `(Phone) item`，当流水线上运来一台显示器时，程序会报什么异常？

> [请在此处写下你的理解]

> 程序会崩溃

>

### 🇨🇳 题目五：跨地区支付网关引擎的模块化（包结构与访问控制）

**\*\*1. 核心代码展示（重点展示 `com.pay.channel` 包下的代码及 `Main` 测试类）：** \*\*

```
java
```

```
package com.pay.channel;
```

```
import com.pay.core.核心支付引擎.PaymentMethod;
```

```
public class 具体支付渠道 {
```

```
 public class WeChatPay implements PaymentMethod{
```

```
 public void pay(double amount)
```

```
 {
```

```
 System.out.println("Processing WeChat Pay for amount: "+amount);
```

```
 }
```



```

 }

}

package com.pay.Main;

import com.pay.core.核心支付引擎.*;

import com.pay.channel.*;

public class Main {

 public static void main(String[] args) {

 具体支付渠道 channel = new 具体支付渠道();

 具体支付渠道.WeChatPay weChatPay = channel.new WeChatPay();

 PaymentGateway.process(weChatPay, 100.0);

 }

}

```

```
```java
```

```
// 请在此处粘贴测试类核心代码
```

```
// 注意：需展示你代码最上方的 package 声明、跨包的 import 语句，以及如何调用网关的过程
```

```
```
```

**\*\*2. 运行结果截图： \*\***

```
底层安全扫描通过
Processing WeChat Pay for amount: 100.0
>
```

**\*\*3. 📄 思考题（必答）：\*\***

如果你在自己独立的测试类里写下一行代码 ``new SecurityCheck()``，编译器会报错吗？  
架构师为什么要把这个类设计成默认（包私有）访问权限，而不是 ``public``？

> 会，为了防止外部修改